

HTTP Desync Attacks: what happened next

HTTP Desync Attacks: what happened next - James Kettle, PortSwigger.

- Published: 2019-10-03
- Original: <<https://portswigger.net/research/http-desync-attacks-what-happened-next>>
- Preserved from: <https://portswigger.net/research/http-desync-attacks-what-happened-next> (live) on 2026-09-16
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

HTTP Desync Attacks: what happened next | PortSwigger Research

HTTP Desync Attacks: what happened next

[image: James Kettle]

James Kettle

Director of Research

[@albinowax](#)

-

Published: **Thursday, 3 October 2019 at 12:54 UTC**

-

Updated: **Tuesday, 20 September 2022 at 08:04 UTC**

-



Last month I published [HTTP Desync Attacks: Request Smuggling Reborn](#). Since then, there's been a range of new developments. While vendors have been deploying fixes and publishing advisories, I've devised new desync techniques exposing yet more systems to attack. I've also adapted the tooling to make it easier to hunt down the dwindling number of vulnerable servers. In this post I'll take a look at what's new, then explore how to handle some common gotchas.

Patches

Akamai deployed a hotfix roughly 48 hours after my presentation - their network of front-end servers now respect [RFC 7230](#) and prioritise chunked Transfer-Encoding by default, meaning that the majority of websites using Akamai are no longer vulnerable. This appears to have been a silent fix with no public advisory or acknowledgement of the vulnerability.

F5 published advisory [K50375550](#) for their BIG-IP servers, in which they suggest several potential workarounds, including configuration hardening and deploying newly released WAF rules. Enabling the protocol compliance enforcement sounds like the most reliable fix.

Nathan Davison [noticed that](#) HAProxy failed to normalise the vertical tab technique included in HTTP Request Smuggler, leaving certain backend servers like gunicorn exposed to attack. They resolved this with release 2.0.6. I hear another popular server is also vulnerable when paired with gunicorn, so a patch for gunicorn itself may be in order.

Finally, Golang published [CVE-2019-16276](#) for their net/http library.

New Techniques

When I initially researched HTTP Desync Attacks I discovered so many vulnerable servers I didn't have time to explore every idea for causing desynchronisation, let alone report every vulnerability. Since publication I've focused on exploring new desync techniques, to

ensure my presentation last week at OWASP Global AppSec Amsterdam has some fresh content.

As usual I tried each of my desync ideas out on my scanning pipeline to identify which ones actually work. The coolest idea that completely failed was suggested to me by [@Zrari Anas](#). HTTP headers are ASCII so you can't apply classic unicode normalization attacks, but you can in theory use extended-ASCII characters, for example:

```
Transfer-Encoding: chùnked
```

Alas I couldn't find any web servers doing this kind of normalization. If you're interested in what else failed to make the cut, feel free to peruse [the commit log](#).

So, what actually worked?

Some research originally done to bypass WAFs can be repurposed for full request smuggling attacks. One successful attack came from lifting a technique from [Steffen Ullrich's research](#) on bypassing IDS:

```
Suricata seems to interpret every value for Transfer-Encoding as chunked, even "chu"
```

You can exploit assorted real systems using this trick - to use it, ensure you've got the latest version of [HTTP Request Smuggler](#) and enable the 'lazygrep' technique.

I also had success using a subtle variation of an existing technique:

```
Transfer-Encoding: \x00chunked
```

The most successful new technique came from WAF-bypass research by Soroush Dalili, [buried in a spreadsheet](#):

```
Foo: bar\r\n\rTransfer-Encoding: chunked
```

This superfluous `\r` enabled me to exploit numerous interesting systems leading to a \$16,500 bounty - and teasingly caused false positives on Google's entire infrastructure (at least, I think they're false positives).

Tooling

During the original research I developed a way to scan for [HTTP Request Smuggling](#) without risk to other users or false-negatives, using a timeout-based heuristic. As it was inference-based, roughly 1% of the results were false positives. Unfortunately as real vulnerabilities get fixed while server behaviours that cause false positives get ignored, this false-positive rate can be expected to steadily increase over time. I've taken a few steps to address this in the latest release of HTTP Request Smuggler (v1.02):

- Added the 'only report exploitable' option which hides situations where desynchronisation is technically possible but not likely to be exploitable
- Added a 'risky mode' option which reduces false positives but may cause dropped requests for other users on vulnerable websites
- Added an extra request to the issue reported for CL.TE findings to help users quickly weed out less-promising findings

Methodology

A few people reported the same problem to me - they'd found a genuine request smuggling vulnerability, but seemed to only be able to exploit themselves, and wanted to know what to do in this situation.

First, make sure `requestsPerConnection` in Turbo Intruder is set to 1. Any higher, and you'll potentially cause a desync between Turbo Intruder and the front-end server, which is useless and effectively a false positive.

Next, recall you can only poison requests that are routed to the same back-end server. Since routing might be based on the request cookie, path, method or any other request property you should start with a 'victim' request that's near-identical to the attack request, then change each value and retry the attack in turn until the victim request resembles a regular GET request sent by another user.

If the regular GET request is still getting poisoned but you can't exploit other users, or the vulnerability is only visible intermittently, one possibility is that the target has multiple front-end servers and only some of them are vulnerable. You can explore this possibility using dig and the Hostname Resolution feature in Burp's Project Options.

Finally, the front-end's connection reuse might be tied to your IP. To explore this, try sending the victim requests from a different IP. I personally test this using regular Intruder, plus an upstream SOCKS proxy powered by an SSH tunnel. Turbo Intruder ignores proxy settings unless you set `Engine=Engine.BURP`.

If you find you're genuinely only able to affect requests from your own IP, the practical impact is limited to exploiting other people on the same corporate network... or direct attacks like the one I demonstrated on New Relic.

Further reading

The reports sent to [PayPal](#) and [NewRelic](#) are now public, as is [an exploit writeup by @memN0ps](#).

Regilero just released [an excellent writeup](#) on some vulnerabilities he found in Apache Traffic Server last year, which includes a Docker image to set up your own vulnerable environment. As always, I advise getting familiar in a safe environment like the [Academy labs](#) before targeting live systems.

You may also be interested in the followup post [Breaking the chains on HTTP Request Smuggler](#).

Finally, I still regard this as a promising topic for further research so you can expect more desync techniques to arrive from myself and others, and I'd encourage everyone to try out their own ideas.

[Request Smuggling](#)

[Back to all articles](#)

